Föreläsning 13

Söndra och härska

Föreläsning 13

- Söndra och härska
- Maximal delsekvens
- Skyline
- Closest pair
- Växel

Söndra och härska (Divide and conquer)

Vi stötte på dessa algoritmer när vi tittade på sortering. Vi ska nu titta på en del andra problem där strategin är effektiv. Låt oss först repetera:

En söndra och härska algoritm bryter ner ett problem i två eller fler subproblem av samma (eller närliggande) typ. Dessa i sin tur brytes också ner i två eller fler subproblem. Algoritmen fortsätter så till dess att subproblemen blivit så små att de är lätta att lösa och kombinerar då dellösningarna till en lösning på hela problemet oftast i samma steg som problemet bröts ner.

- Divide – bryt ner problemet i mindre delar
- Conquer – lös varje del (genom att bryta ner ytterligare)
- Combine – kombinera dellösningarna till en lösning av hela problemet

Maximal delsekvens

Givet en sekvens tal (positiva och negativa) söker vi summan av den delsekvens som ger största summan.

24, 35, -87, **21, 80, -12, 32**, -90, 45, 10

Hur skulle en söndra och härska algoritm se ut?

Maximal delsekvens – söndra och härska

Indata: a – array med sekvensen
Utdata: maximala summan
Algoritm

```
maxSum(a,left,right)
om left=right return max(0,a[left])
center=(left+right)/2
sum=0,maxLeft=0
för i=center ner till left
        sum=sum+a[i]
        maxLeft=max(maxLeft,sum)
sum=0,maxRight=0
för i=center+1 upp till right
        sum=sum+a[i]
        maxRight=max(maxRight,sum)
maxÖverMitt=maxLeft+maxRight
return max(maxÖvermitt,maxSum(a,left,center),maxSum(a,center+1,right))
```

största summa över mitten

Analys

I princip får vi $T(n) = 2T(n/2) + n$ och $T(1) = 1$

vilket ger $O(n\log n)$ (vilket vi inte visar)

Smartare algoritm

Indata: a – array med sekvensen (längd n)

Utdata: maximala summan

Algoritm

maxSum(a)

```
maxSumma=0
maxLokal=0
för i = 0 till n-1
        maxLokal=max(maxLokal+a[i],0)
        maxSumma=max(maxLokal,maxSumma)
return maxSumma
```

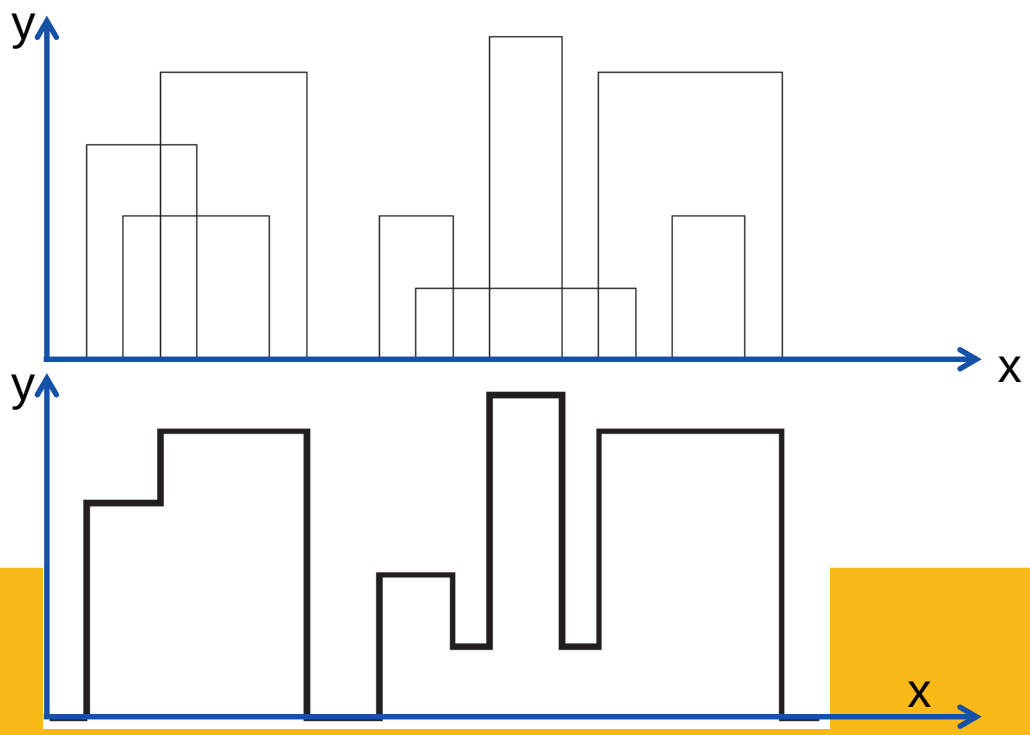
O(n) - ☺ något enklare… men inte söndra och härska

Skyline - problemet

Givet huskroppars koordinater vad blir deras skyline?

Det här problemet löses inte bara elegant utan också effektivt med en söndra och härska algoritm.

Problemet är generellare än det tycks och handlar om alla problem där man ska optimera utifrån två dimensioner (ex billigast hyra kontra närmast havet) och då vill hitta de kandidater för varje värde på den ena dimensionen är bäst i den andra dimensionen.



En naiv lösning

Vi ska bygga upp en skyline av n stycken hus. Vi gör detta rekursivt genom att lägga till ett hus till en skyline för n-1 hus.

Slutvillkoret blir att ta fram en skyline för ett hus vilket är trivialt.

Analys
Vi kommer att behöva lägga till n-1 hus till en skyline. När vi ska lägga till ett hus kommer vi behöva gå igenom i snitt hälften av punkterna för att hitta ”rätt platts”. Detta ger

$O(1)+O(2)+\ldots+O(n-2)+O(n-1) = O(n^2)$

Indata och utdata

Om vi låter alla hus stå på marken (y=0) behöver vi tre koordinater för att beskriva ett hus (x_1, y, x_2).

Indata är en array av sådana punkter ordnade efter första x-värdet

Om vi börjar i origo kan vi beskriva en skyline med en sekvens av alternerande x- och y-värden där vi börjar med x (eftersom endast en i taget varierar):

Utdata: [x1,y1,x2,y2,x3,y3,…]

betyder då punkterna

(x1,0),(x1,y1),(x2,y1),(x2,y2),(x3,y2),(x3,y3),…

Söndra och härska algoritm

- Dela upp indata i två arrayer med husvärden.

- Beräkna en skyline för varje del.
  Detta görs då genom att dela upp indata i två arrayer. Slutvillkoret är att när vi har ett hus levererar vi en skyline.

- Foga samman dessa två skyline till en.
  Denna del är den svåraste men är på det stora hela en merge där vi ska ta bort onödiga värden.

Analys

I varje steg halverar vi datamängden med uppdelningen. När vi sammanfogar två lösningar går vi igenom antalet punkter och väljer rätt:

$$T(1) = 1 \text{ och } T(n) = 2T(n/2) + n$$

vilket ger $O(n\log n)$

Closest pair

Givet n punkter i planet ska vi hitta de två som ligger närmast varandra.

Med brutal kraft beräknar vi för varje punkt avståndet till varje annan punkt och väljer de punkter som har kortast avstånd. Detta ger $n(n-1)/2 = O(n^2)$.

Hur skulle en söndra och härska algoritm fungera? Vad blir problematiskt.

Naiv: Söndra och härska - algoritm

Algoritmen är formulerad så att den endast bestämmer avståndet. Vill man få ut punkterna får man se till att spara detta.

Input: a[n] – array med punkterna p=(x,y), n – antal punkter

Sortera punkterna i a efter x-koordinaten och anropa
d = hittaNärmsta(a,0,n-1)

<u>hittaNärmsta(a,vänster,höger)</u>

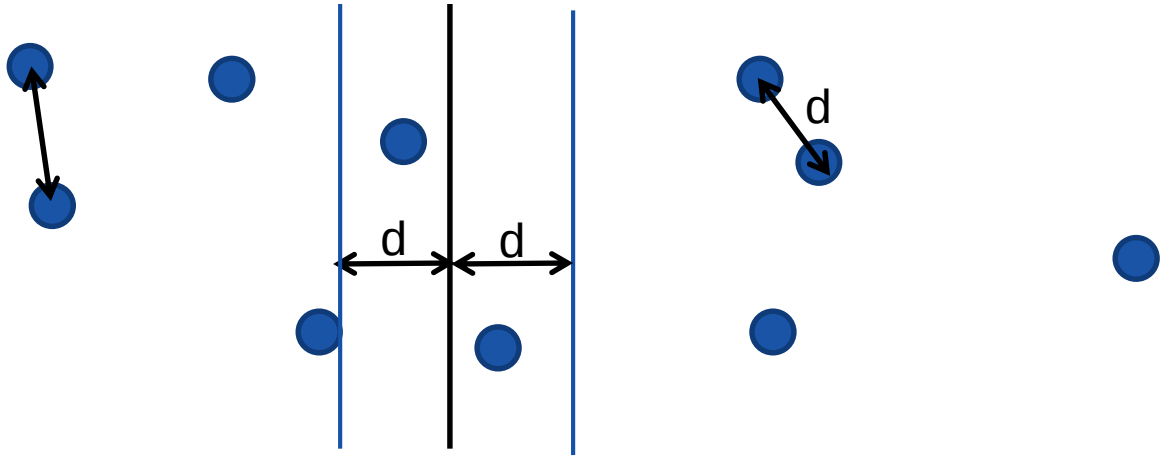
```
mitt = (vänster+höger)/2
return min(hittaNärmsta(a,vänster,mitt),hittaNärmsta(a,mitt+1,höger))
```

Men kanske närmsta paret har en punkt på varje sida om mitten?

Combine delen måste förfinas till att kontrollera punktpar över mitten med potentiellt kortare avstånd än vi fått ovan.

Närmsta par över mitten

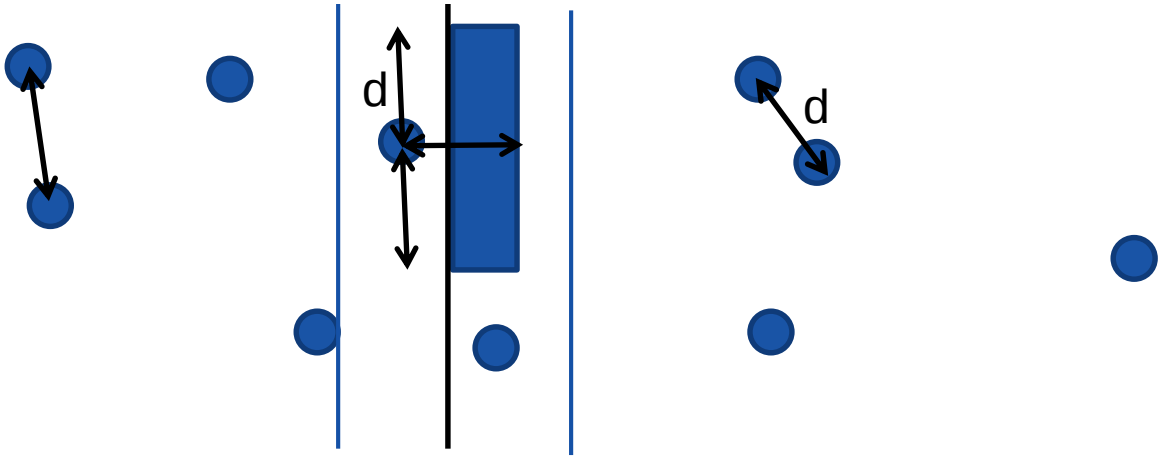
Om vi skulle kontrollera alla par över mitten skulle vi behöva beräkna (n/2)(n/2) stycken och vi är tillbaka till $O(n^2)$. Det behöver vi inte göra. Vi behöver endast kontrollera de punkter som ligger nära mitten:



Den svarta linjen beräknas såsom medelvärdet av x-värdena hos punkterna a[mitt] och a[mitt+1]

Bättre

Vi skulle fortfarande kunna riskera att få kontrollera många punkter (i värsta fall hälften mot hälften) men med följande tillägg kan man visa att vi behöver kontrollera maximalt sex stycken till varje punkt i intervallet. För varje punkt i intervallet kontrollera avståndet endast mot de punkter som befinner sig maximalt på avståndet d i y- led och avståndet d i x-led.



Genom att sortera aktuella punkter efter y inom intervallet och genom att använda en effektiv merge kan man få ner algoritmen till O(nlogn) inklusive sortering.

Växlingsproblemet med söndra och härska

Vad är det minsta antalet mynt och sedlar som krävs för att ge växel för ett givet belopp?

Vår giriga algoritm fungerade inte för godtyckliga valutor. Om man ska designa en algoritm som gör det passar en söndra och härska algoritm mycket bra.

Algoritm

```
Input:    belopp kr i växel
          mynt – array med de n olika valörerna
växel(belopp)
          För i = 0 till n-1
                    Om mynt[i]=belopp return 1
          bäst=100000
          För i = 0 till n-1
                    Om mynt[i]<belopp
                              bäst=min(bäst,1+växel(belopp-mynt[i]))
          return bäst
```

Med dynamisk programmering kommer vi att kunna effektivisera den.